YILDIZ TECHNICAL UNIVERSITY

Faculty of Chemistry and Metallurgy

Department of Mathematical Engineering

COURSE PROJECT FINAL REPORT

Private Lesson Management System

(PLMS)

MTM4692 Applied SQL — 2025-2026 Spring Semester

Project Group:	Team Members (5 Members)
Database Platform:	SQLite via DB Browser for SQLite
Repository:	github.com/KalenderCakmak/PLMS-Database-Project
Submission Date:	June 8, 2026

1. Introduction and Problem Statement

The Private Lesson Management System (PLMS) is a relational database architecture developed to resolve the operational, administrative, financial, and academic challenges faced by independent private tutors and boutique-scale tutoring institutions. Currently, many independent educators and small tutoring facilities still rely on traditional, manual methods—such as paper logs, physical planners, or fragmented, unlinked spreadsheets—to manage their daily workflows. This outdated and non-integrated approach hinders institutional scaling and creates several critical operational problems:

- **Operational Inefficiency & Scheduling Conflicts:** Attempting to manually synchronize the schedules of multiple students and instructors frequently results in logistical errors, such as double-bookings or overlapping session hours. Without a centralized architecture and constraint verification mechanisms, time allocation optimization becomes virtually impossible.
- **Financial Opacity & Revenue Loss:** The absence of a structured financial tracking model makes real-time tracking of income versus outstanding receivables difficult. Manual validation of pending balances, delayed payments, or prepaid block packages leads to uncollected fees, overlooked balances, and major accounting discrepancies.
- **Lack of Progress Monitoring & Academic Continuity:** Traditional archiving models fail to log historical session data and performance metrics in a structured manner. When information regarding which topics were covered, when they were completed, and where the learning gaps lie is not analytically traceable, the overall educational efficacy drops significantly.
- **Data Security & Scalability Constraints:** Physical records and disconnected spreadsheets are highly vulnerable to permanent loss, degradation, and unauthorized access. As the client baseline expands, manual processing scales up complexity exponentially, preventing the business from growing professionally.

2. Project Objectives

The overarching objective of this project is to implement a robust, scalable, and highly defensive relational database system that automates core administrative and academic private tutoring processes while maintaining absolute data integrity. The system targets the following specific milestones:

1. **Centralized Information Management:** Consolidating student profiles, tutor specializations, and granular course definitions into a singular, normalized database schema to eradicate data fragmentation.
2. **Conflict-Free Appointment Architecture:** Establishing a scheduling framework that prevents temporal double-bookings, optimizing resource allocation for both educators and students.
3. **Rigorous Financial Auditing:** Automating payment states (Paid/Pending), accurately calculating real-time revenue matrices, and ensuring transparency across billing profiles.
4. **Data-Driven Pedagogical Tracking:** Enabling tutors to attach progress logs to appointments, making student historical academic evolution transparently traceable.

3. Database Schema and Relational Design

To execute the core administrative criteria seamlessly, a normalized database layout composed of 6 interconnected tables has been engineered. The architectural role and structural constraints of each table are summarized below:

Table Name	Description / Operational Role	Keys and Integrity Constraints
Students	Archives student identities, persistent contact profiles, and academic grade milestones.	PK: Student_ID UNIQUE: Contact_Info
Tutors	Stores instructor demographics, specialized areas (e.g., Mathematics, Physics), and validated contact rows.	PK: Tutor_ID UNIQUE: Phone_Number
Lessons	Defines specific subject topics offered and enforces global baseline hourly pricing tiers.	PK: Lesson_ID UNIQUE: Subject_Name CHECK: Hourly_Rate > 0
Appointments	The centralized operational hub. Maps concrete transactional links connecting a student, an instructor, and a subject to a explicit date and time.	PK: Appointment_ID FK: Student_ID, Tutor_ID, Lesson_ID
Payments	Tracks all billing occurrences, historical transaction balances, and current processing clearances.	PK: Payment_ID FK: Student_ID CHECK: Amount > 0, Status
Progress_Notes	Captures post-session qualitative text logs detailing topics covered and subjective developmental reviews.	PK: Note_ID FK: Appointment_ID

4. SQL Implementation and Data Definition (DDL)

The planned architecture was fully materialized utilizing SQLite standards. Referential integrity is strictly maintained through structural constraints, utilizing mandatory foreign key cascading parameters and type verification checks to guarantee defensive data ingestion.

4.1 Data Definition Language (DDL) Script

```
-- Table Structure: Students
CREATE TABLE Students (
    Student_ID   INTEGER PRIMARY KEY AUTOINCREMENT,
    Name         TEXT    NOT NULL,
    Surname      TEXT    NOT NULL,
    Contact_Info TEXT    UNIQUE,
    Grade_Level  TEXT    NOT NULL
);

-- Table Structure: Tutors
CREATE TABLE Tutors (
    Tutor_ID       INTEGER PRIMARY KEY AUTOINCREMENT,
    Name           TEXT    NOT NULL,
    Surname        TEXT    NOT NULL,
    Specialization TEXT    NOT NULL,
    Phone_Number   TEXT    UNIQUE
);

-- Table Structure: Lessons
CREATE TABLE Lessons (
    Lesson_ID    INTEGER PRIMARY KEY AUTOINCREMENT,
    Subject_Name TEXT    NOT NULL UNIQUE,
    Hourly_Rate  REAL    NOT NULL CHECK (Hourly_Rate > 0)
);

-- Table Structure: Appointments (Central Junction Table)
CREATE TABLE Appointments (
    Appointment_ID INTEGER PRIMARY KEY AUTOINCREMENT,
    Student_ID     INTEGER NOT NULL,
    Tutor_ID       INTEGER NOT NULL,
    Lesson_ID      INTEGER NOT NULL,
    Date           TEXT    NOT NULL,
    Time           TEXT    NOT NULL,
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID),
    FOREIGN KEY (Tutor_ID)   REFERENCES Tutors(Tutor_ID),
    FOREIGN KEY (Lesson_ID)  REFERENCES Lessons(Lesson_ID)
);

-- Table Structure: Payments
CREATE TABLE Payments (
    Payment_ID   INTEGER PRIMARY KEY AUTOINCREMENT,
    Student_ID   INTEGER NOT NULL,
    Amount       REAL    NOT NULL CHECK (Amount > 0),
    Payment_Date TEXT    NOT NULL,
    Status       TEXT    NOT NULL CHECK (Status IN ('Paid', 'Pending')),
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)
);

-- Table Structure: Progress_Notes
CREATE TABLE Progress_Notes (
    Note_ID        INTEGER PRIMARY KEY AUTOINCREMENT,
    Appointment_ID INTEGER NOT NULL,
    Description    TEXT    NOT NULL,
```

```
    FOREIGN KEY (Appointment_ID) REFERENCES Appointments(Appointment_ID)
);
```

5. Virtual Reporting Views

To encapsulate complex relational joins, accelerate recurring query processes, and simplify administrative reporting metrics, 3 logical views (Views) were compiled directly over the schema framework.

VIEW 1: Daily Operational Calendar (Daily_Schedule)

Translates abstract system identifiers into human-readable text configurations to extract real-time daily schedules.

```
CREATE VIEW Daily_Schedule AS
SELECT
    A.Appointment_ID,
    S.Name || ' ' || S.Surname AS Student,
    T.Name || ' ' || T.Surname AS Tutor,
    L.Subject_Name             AS Subject,
    A.Date,
    A.Time
FROM Appointments A
INNER JOIN Students S ON A.Student_ID = S.Student_ID
INNER JOIN Tutors   T ON A.Tutor_ID   = T.Tutor_ID
INNER JOIN Lessons  L ON A.Lesson_ID  = L.Lesson_ID;
```

VIEW 2: Financial Transaction Matrix (Payment_Summary)

Abstracts cashflow records to evaluate general transactional amounts, execution dates, and real-time validation flags per student profile.

```
CREATE VIEW Payment_Summary AS
SELECT
    S.Name || ' ' || S.Surname AS Student,
    P.Amount,
    P.Payment_Date,
    P.Status
FROM Payments P
INNER JOIN Students S ON P.Student_ID = S.Student_ID;
```

VIEW 3: Academic Progression Ledger (Student_Progress)

Provides deep chronological oversight, matching pedagogical text logs directly to structural student profiles and target subject definitions.

```
CREATE VIEW Student_Progress AS
SELECT
    S.Name || ' ' || S.Surname AS Student,
    L.Subject_Name             AS Subject,
    A.Date,
    PN.Description
FROM Progress_Notes PN
INNER JOIN Appointments A ON PN.Appointment_ID = A.Appointment_ID
INNER JOIN Students     S ON A.Student_ID      = S.Student_ID
INNER JOIN Lessons      L ON A.Lesson_ID       = L.Lesson_ID;
```

6. Advanced SQL Analytical Queries

To support complex administrative reporting and decision-making, advanced data retrieval strategies—including aggregation functions, conditional group filters, and subqueries—were engineered.

Query 1: Dynamic Extraction of Schedules via explicit Date Boundaries

```
SELECT * FROM Daily_Schedule WHERE Date = '2026-05-01';
```

Query 2: Net Monthly Financial Inflow Computations (Enforcing 'Paid' constraint blocks)

```
SELECT SUM(Amount) AS Total_Revenue
FROM Payments
WHERE Status = 'Paid' AND Payment_Date LIKE '2026-05%';
```

Query 3: Instructor Performance Load Evaluations (Workload Distribution Metrics)

```
SELECT
    T.Name || ' ' || T.Surname AS Tutor,
    COUNT(*) AS Lesson_Count
FROM Appointments A
INNER JOIN Tutors T ON A.Tutor_ID = T.Tutor_ID
GROUP BY T.Tutor_ID
ORDER BY Lesson_Count DESC;
```

Query 4: Primary Consumer Tracking via Correlated Subqueries

```
SELECT
    Name || ' ' || Surname AS Student,
    (SELECT COUNT(*) FROM Appointments WHERE Student_ID = S.Student_ID) AS Total_Lessons
FROM Students S
ORDER BY Total_Lessons DESC
LIMIT 1;
```

Query 5: Identifying High-Load Faculty Nodes (HAVING Clause Isolation)

```
SELECT
    T.Name || ' ' || T.Surname AS Tutor,
    COUNT(*) AS Lesson_Count
FROM Appointments A
INNER JOIN Tutors T ON A.Tutor_ID = T.Tutor_ID
GROUP BY T.Tutor_ID
HAVING COUNT(*) > 2;
```

Query 6: Outlier Isolation and Retention Risk Audits (LEFT JOIN Exception Identification)

```
SELECT S.Name || ' ' || S.Surname AS Student
FROM Students S
LEFT JOIN Appointments A ON S.Student_ID = A.Student_ID
WHERE A.Appointment_ID IS NULL;
```

7. Performance Tuning and Indexing Architecture

To guarantee sub-millisecond execution patterns even when database scale breaches tens of thousands of rows, query search paths have been systematically optimized. B-Tree-structured indices were introduced across heavily targeted foreign keys and status flags:

```
-- Optimizes individual student lookup trees and relational join parameters
CREATE INDEX idx_appointments_student ON Appointments(Student_ID);

-- Minimizes sorting costs for chronological calendar range sweeps
CREATE INDEX idx_appointments_date    ON Appointments(Date);

-- Speeds up financial ledger lookups and individual accounting auditing balances
CREATE INDEX idx_payments_student     ON Payments(Student_ID);

-- Accelerates status filtering for fast liquidity and balance reviews
CREATE INDEX idx_payments_status      ON Payments(Status);
```

This indexing layout transforms flat linear scans into precise log-linear lookups, maintaining a strict *O(log N)* execution cost ceiling across scale.

8. Defensive Design and Constraint Verification Modeling

The computational resilience and structural enforcement rules of the system were systematically tested inside the DB Browser for SQLite workspace. The database engine successfully blocked invalid data configurations, as detailed below:

Violation Simulation 1: Negative Financial Value Entry Attempt

```
INSERT INTO Payments (Student_ID, Amount, Payment_Date, Status) VALUES (1, -100.0,
'2026-05-15', 'Paid');
[REJECTED] Result: CHECK constraint failed: Amount > 0
```

Analysis: The core relational engine proactively rejects invalid cash flow metrics through the active enforcement of the `CHECK (Amount > 0)` rule.

Violation Simulation 2: Out-of-Bounds Status State Input

```
INSERT INTO Payments (Student_ID, Amount, Payment_Date, Status) VALUES (1, 300.0,
'2026-05-15', 'Cancelled');
[REJECTED] Result: CHECK constraint failed: Status IN ('Paid', 'Pending')
```

Analysis: Business state workflows are strongly normalized. Entry points that do not match permitted attributes are discarded via the `CHECK (Status IN (...))` parameters.

Violation Simulation 3: Duplicate E-Mail Identity Input Conflict

```
INSERT INTO Students (Name, Surname, Contact_Info, Grade_Level) VALUES ('Test', 'User',
'ayse.kaya@gmail.com', '9th Grade');
[REJECTED] Result: UNIQUE constraint failed: Students.Contact_Info
```

Analysis: Enforcing strict uniqueness across identity strings prevents multi-profile collisions and structural redundancies via the `UNIQUE` entity criteria.

9. Conclusion

The engineered Private Lesson Management System (PLMS) satisfies all foundational administrative, analytical, and security criteria. Real-world structural benchmarks, operational tests, and index tuning verification reveal an enterprise-grade relational structure that remains exceptionally safe, secure, and performant under load. This architecture effectively demonstrates how transitioning educational ecosystems from legacy spreadsheets to unified database layouts maximizes tracking transparency and operational agility.

10. Project Repository

The complete open-source database script codebase, documentation files, and verification assets are centrally hosted and auditable via the public engineering link below:

Official GitHub Repository:

https://github.com/KalenderCakmak/PLMS-Database-Project